

Supply Chain Security Roadmap

Status: Draft

Purpose

Bayview should assume that a software supply chain attack will eventually affect the organization or one of its dependencies. That assumption changes the goal from only trying to predict the next compromised npm, PyPI, Maven, NuGet, container, or operating system package to also reducing the blast radius when a compromised package executes.

The goal of this roadmap is to make safe package use, isolated development, governed build environments, and fast incident response part of the normal development lifecycle.

This roadmap expands on [Supply Chain Security](#) and [Citizen Developers and Vibe Coders](#).

Threat Context

Supply chain attacks are no longer limited to obscure packages or simple typosquatting. Modern campaigns compromise trusted packages, steal developer credentials, abuse CI/CD workflows, publish malicious child dependencies, and spread through legitimate package manager behavior.

The Shai-Hulud campaign is a useful planning example. Public reporting described a May 2026 Mini Shai-Hulud wave affecting npm and PyPI packages, including a May 11, 2026 wave against package ecosystems used by developers. On May 12, 2026, researchers reported that weaponized Shai-Hulud code was publicly released online, increasing the likelihood of copycat attacks and variants. Additional public reporting on May 19, 2026 described hundreds of additional npm package versions being published in a later wave.

Bayview should treat this as a warning about attacker repeatability. It is not realistic to know which dependency will be compromised next. It is realistic to control where packages come from, where package install scripts run, what secrets are available in those environments, and how quickly the SOC and AppSec teams can determine whether Bayview is affected.

Target State

- Developers, CI/CD pipelines, containers, and deployment workflows use approved package sources instead of direct public downloads.
- Package manifests and lockfiles are committed to approved source control and scanned.
- New packages, AI-suggested packages, major version upgrades, and unusual dependencies are reviewed before production use.
- Dependency cooldowns prevent automatic adoption of brand-new package versions until enough time has passed for malicious releases to be detected.
- High-risk development runs in sandboxed environments such as GitHub Codespaces, stripped-down virtual machines, dedicated build containers, or hardened CI runners.
- Local laptop development is restricted or prohibited for high-risk production-bound projects once a usable sandboxed development path exists.

- Developers have a supported "happy path" for development environments, tools, package sources, authentication, debugging, and deployment.
- Containerized applications use approved minimal base images and include only necessary runtime packages.
- Hardened package providers or cleanroom-built package sources, such as Seal Security, Chainguard, or equivalent approved services, are evaluated for critical applications and difficult-to-patch vulnerabilities.
- The SOC, AppSec, Cloud Security, and engineering teams can quickly determine whether Bayview uses a compromised package, version, container image, infrastructure pattern, or CI/CD workflow.
- Credential rotation, indicator hunting, package removal, rebuild, redeploy, and exception handling are documented and practiced.

Guiding Principles

- **Assume compromise.** The roadmap should reduce impact when a package, maintainer account, registry, or build workflow is compromised.
- **Reduce available secrets.** Package installation and build steps should not have broad access to developer API keys, cloud credentials, personal files, production tokens, or administrative tools.
- **Make the secure path usable.** Developers need approved environments with the tools they need. If sandboxed development is painful, teams will route around it.
- **Prefer stable dependencies.** Teams should avoid unnecessary packages, newly published packages, abandoned packages, and bleeding-edge versions unless there is a clear reason.
- **Control package sources.** Direct downloads from public registries, GitHub release URLs, shell installer commands, and random vendor sites should be exceptions, not the default.
- **Fix without breaking where possible.** Backported fixes and hardened packages can reduce vulnerability exposure when a direct upstream upgrade would cause application risk.
- **Keep citizen-developed code visible.** Scripts, automations, vibe-coded tools, and low-code projects can introduce the same dependency and credential risks as formal applications.

Tools Breakdown

These tools are complementary, not interchangeable. Artifactory and Nexus are repository managers that help control where packages and artifacts come from. Chainguard and Seal Security reduce vulnerability and compromise exposure by changing what Bayview runs or consumes. Socket.dev should also be considered as a package-risk firewall and developer-workflow control, especially for blocking malicious or suspicious packages before they reach developer workstations or CI/CD.

Tool	Category	What It Actually Does	What It Defends Against	Best Fit in This Roadmap	What It Does Not Replace
JFrog Artifactory	Universal artifact and package repository manager	Hosts internal packages and build artifacts, proxies and caches public registries, supports many package and container	Direct public downloads, unmanaged package sources, dependency confusion risk when routing is configured correctly,	Primary approved package source for developers and CI/CD, especially where Bayview wants one enterprise repository	SCA tools, malicious package judgment by itself, source code review, hardened base images,

Tool	Category	What It Actually Does	What It Defends Against	Best Fit in This Roadmap	What It Does Not Replace
		formats, provides metadata, access control, and auditability for artifact use.	untracked artifact movement, and lack of visibility into what developers and CI/CD are pulling.	pattern across npm, PyPI, Maven, NuGet, containers, and other formats.	or incident response.
Sonatype Nexus Repository	Universal artifact and package repository manager	Provides hosted, proxy, and grouped repositories for common package ecosystems so teams can centralize dependency access instead of downloading directly from public registries.	Direct public downloads, unmanaged package sources, dependency confusion risk when routing is configured correctly, inconsistent repository use, and limited auditability of package consumption.	Alternative primary package source to Artifactory, especially if Sonatype tooling or Maven-heavy workflows are already preferred.	SCA tools, malicious package judgment by itself, source code review, hardened base images, or incident response.
Chainguard	Hardened container image provider	Provides minimal, hardened container images built for reduced attack surface, frequent rebuilds, supply chain metadata, and lower vulnerability counts compared with broad general-purpose base images.	Bloated runtime images, stale OS packages, unnecessary shells and tools in production containers, excessive CVE noise, and opaque base image provenance.	Approved base image catalog for Tier 1 services and containerized workloads where Bayview wants fewer OS packages, fewer runtime tools, SBOM support, and faster base image remediation.	A repository manager, application dependency approval, package ingestion firewall, or complete replacement for application testing.
Seal Security	Open source vulnerability remediation and	Produces patched versions of vulnerable open source	Known CVEs in direct and transitive dependencies,	Legacy, hard-to-upgrade, regulated, or SLA-sensitive	A repository manager, install-time malware

Tool	Category	What It Actually Does	What It Defends Against	Best Fit in This Roadmap	What It Does Not Replace
	backported patch provider	dependencies, OS packages, and container components so teams can remediate CVEs without always taking upstream upgrades or breaking changes.	legacy or EOL components with no easy upgrade path, vulnerable OS packages, and remediation delays caused by breaking upstream upgrades.	applications where direct dependency upgrades are risky but critical and high vulnerabilities still need fast remediation.	blocking, source control protection, CI/CD hardening, or normal upgrade planning.

Artifactory and Nexus should generally be treated as a platform selection decision: Bayview may not need both as primary enterprise repository managers unless different business units already depend on them. Chainguard and Seal Security can be evaluated independently because they solve different problems: hardened runtime inputs versus backported vulnerability remediation. Socket.dev or an equivalent tool belongs in the evaluation as a detection and policy layer that can sit in front of developers and CI/CD, upstream of Artifactory or Nexus, or between internal repository tiers.

Adoption Phases

Phase 0: Inventory and Risk Tiering

Suggested timing: Weeks 0-4

Objectives

- Understand where Bayview uses third-party code, where package installation occurs, and where secrets could be exposed.
- Prioritize high-risk applications and developer workflows.

Actions

- Inventory repositories, package ecosystems, package managers, lockfiles, container images, CI/CD systems, and build runners.
- Identify projects that use npm, PyPI, Maven, Gradle, NuGet, Go modules, RubyGems, container registries, operating system package managers, or shell-based installers.
- Identify where package installation runs today: developer laptops, Codespaces, VMs, CI/CD runners, containers, production hosts, or shared servers.
- Inventory secrets commonly available during development and build, including GitHub tokens, npm/PyPI tokens, cloud keys, SSH keys, kubeconfigs, API keys, and service account credentials.
- Tier projects by supply chain risk:
 - **Tier 1:** Production-bound, internet-facing, regulated, customer-data, loan-data, privileged, cloud-infrastructure, or broad internal-use systems.
 - **Tier 2:** Internal applications, shared libraries, scheduled jobs, APIs, data pipelines, and department tools.

- **Tier 3:** Developer utilities, prototypes, scripts, citizen-developed tools, and lower-risk automations.
- Identify citizen-developed and AI-assisted projects that are not yet in approved source control.

Exit Criteria

- Package ecosystem inventory exists for Tier 1 and Tier 2 projects.
- High-risk local development and build workflows are identified.
- Known unmanaged or citizen-developed projects have owners or an intake path.
- Initial metrics are available for approved package source usage, lockfile coverage, and Wiz SCA coverage.

Phase 1: Package Source Governance

Suggested timing: Weeks 4-10

Objectives

- Route developers and build systems through approved package sources.
- Create enough visibility and control to know which package versions are in use.

Actions

- Select the enterprise package source pattern for each ecosystem, such as JFrog Artifactory, Nexus, hardened package providers, cloud-native package registries, or approved mirrors.
- Evaluate whether Socket.dev or an equivalent package-risk firewall should sit upstream of the repository manager, downstream in developer and CI/CD workflows, or both.
- Configure package managers to use approved repositories, proxies, mirrors, or hardened providers for npm, PyPI, Maven, NuGet, Go, container, and operating system packages where practical.
- Block or alert on direct public package downloads in CI/CD for Tier 1 projects.
- Require source-controlled manifests and lockfiles for production-bound repositories.
- Define a lightweight new package request process, including package name, ecosystem, version, business purpose, license, maintainer, alternatives considered, transitive dependencies, and whether the package was suggested by an AI tool.
- Define stable version guidance so teams do not automatically pull bleeding-edge versions unless justified.
- Implement dependency cooldowns for high-risk ecosystems so newly published versions are delayed before use.
- Begin recording package approval decisions in a system that AppSec, platform teams, and developers can find.

Exit Criteria

- Tier 1 projects use approved package sources or have documented exceptions.
- New package request process exists and is usable by developers.
- Dependency cooldown policy is defined for npm and at least one additional ecosystem.
- Decision is recorded on whether to pilot Socket.dev or an equivalent package-risk firewall for npm, PyPI, and CI/CD workflows.
- Direct public downloads are visible in Tier 1 CI/CD.

Phase 2: Sandboxed Development Happy Path

Suggested timing: Weeks 8-18

Objectives

- Reduce the blast radius of compromised packages by moving risky development and package installation away from broad-access laptops.
- Give developers an approved environment that does not slow normal work to a crawl.

Actions

- Pilot GitHub Codespaces, stripped-down VMs, sandboxed build containers, or equivalent isolated environments with Tier 1 teams.
- Build standard development environment images that include approved package manager configuration, IDE support, debugging tools, language runtimes, certificate trust, and deployment tooling.
- Define how approved tools are requested, installed, patched, and removed in sandboxed environments.
- Limit secrets available in sandboxed environments to the minimum required for the project.
- Prefer short-lived credentials, scoped tokens, and environment-specific access over long-lived personal API keys.
- Restrict package installation and build steps from accessing unrelated local files, personal directories, broad cloud credentials, or administrative tokens.
- Create a phased strategy to restrict or prohibit local laptop development for Tier 1 projects after the sandboxed happy path is proven.
- Define exception handling for emergency fixes, offline work, specialized hardware, or workflows that cannot yet run in the sandbox.

Exit Criteria

- At least one Tier 1 team can perform normal development in the approved sandboxed environment.
- Required developer tools can be installed through a documented process.
- Secrets available during development are reduced and scoped.
- Policy direction is approved for restricting high-risk local laptop development.

Phase 3: Hardened Packages and Minimal Containers

Suggested timing: Weeks 12-24

Objectives

- Reduce exposure from compromised dependencies, vulnerable packages, bloated container images, and difficult-to-upgrade software.
- Standardize safe runtime patterns for applications.

Actions

- Evaluate hardened package and container providers such as Seal Security, Chainguard, or equivalent approved services.
- Identify use cases where cleanroom-built, hardened, or backported packages are most valuable:
 - Critical vulnerabilities with no safe upstream upgrade path.

- Legacy applications that cannot quickly absorb breaking changes.
- Production services with strict remediation SLAs.
- Base images with recurring vulnerability debt.
- Document the limitation that hardened package services can reduce dependency compromise and vulnerability exposure, but they do not eliminate the need to protect source control, CI/CD, maintainer credentials, and build workflows from tampering.
- Encourage containerization for applications where it improves runtime consistency and isolation.
- Publish approved base image guidance, favoring minimal images such as Alpine, distroless, Chainguard Images, or other approved hardened images when compatible with the application.
- Remove unnecessary shells, package managers, build tools, network tools, and debugging utilities from production runtime images where practical.
- Require container image definitions, manifests, and lockfiles to be stored in source control.
- Add container scanning and base image update expectations to the standard AppSec process.

Exit Criteria

- Hardened package and container provider evaluation is complete.
- Approved base image catalog or guidance exists.
- Tier 1 containerized services have a plan to move to approved minimal or hardened base images.
- Critical vulnerability remediation process includes backported or hardened package options when appropriate.

Phase 4: Enforcement and Engineering Controls

Suggested timing: Months 5-8

Objectives

- Move from guidance to enforceable controls for high-risk projects.
- Prevent unsafe package sources and unmanaged dependencies from reaching production.

Actions

- Require Tier 1 CI/CD pipelines to use approved package sources.
- Add Wiz SCA, secrets, IaC, container, and CI/CD posture checks where supported.
- Block new critical dependency findings, exposed secrets, and direct public download patterns in Tier 1 production-bound pipelines after policy tuning.
- Require review for new direct dependencies, major version upgrades, newly published packages, and AI-suggested packages.
- Where a package-risk firewall is selected, tune and enforce policies for known malware, high-confidence suspicious behavior, typosquats, dependency confusion, and direct public downloads.
- Require source-controlled manifests and lockfiles before release.
- Enforce branch protection or pipeline controls so dependency and package-source checks cannot be silently skipped.
- Create time-bound exceptions for packages, versions, sources, or development workflows that cannot yet meet the standard.
- Report policy violations to engineering leadership and repository owners.

Exit Criteria

- Tier 1 repositories enforce approved package source usage or have approved exceptions.
- Tier 1 releases cannot proceed with unreviewed critical package risk without exception.
- Lockfile and manifest coverage is measurable.
- Wiz and package source data can be mapped to application and team ownership.

Phase 5: Reactive Defense and Incident Readiness

Suggested timing: Months 4-8 and ongoing

Objectives

- Make Bayview fast at answering: "Are we affected by this package, version, campaign, indicator, or behavior?"
- Reduce response time when a compromised dependency reaches a developer environment, build runner, or application.

Actions

- Create a supply chain incident runbook owned by SOC, AppSec, Cloud Security, Platform Engineering, and affected application teams.
- Use Wiz Code, Wiz Mika or equivalent investigation capabilities, package source data, source control history, lockfiles, CI/CD logs, endpoint telemetry, and cloud logs to determine exposure.
- Define standard investigation questions:
 - Are we using the compromised package or version?
 - Was it installed on a developer laptop, sandboxed environment, CI/CD runner, container image, or production host?
 - What secrets were available in that environment?
 - Did the environment contact known suspicious hostnames or IP addresses?
 - Were suspicious files written to disk?
 - Did build scripts, install hooks, or package imports execute unexpected code?
 - Were repositories, workflows, package publishing tokens, or cloud credentials modified?
- Create a credential rotation playbook for GitHub, package registries, cloud providers, CI/CD systems, SSH keys, API keys, service accounts, and developer tokens.
- Define when affected environments must be destroyed and rebuilt rather than cleaned in place.
- Preserve evidence from package manifests, lockfiles, CI/CD logs, endpoint telemetry, network logs, and affected environments.
- Run tabletop exercises using a Shai-Hulud-style scenario where a trusted package steals developer and CI/CD secrets.

Exit Criteria

- SOC and AppSec can answer exposure questions for Tier 1 projects within an agreed response window.
- Credential rotation paths are documented and tested.
- IoC hunting covers hostnames, IP addresses, filesystem artifacts, build logs, and unusual package install behavior.
- A supply chain tabletop has been completed and action items are tracked.

Phase 6: Enterprise Expansion and Optimization

Suggested timing: Months 8-12 and ongoing

Objectives

- Expand controls beyond the highest-risk teams.
- Improve developer experience while steadily reducing attack surface.

Actions

- Expand approved package source requirements to Tier 2 repositories.
- Expand sandboxed development requirements based on data sensitivity, credential exposure, and production adjacency.
- Bring citizen-developed and vibe-coded projects into source control, scanning, package governance, and ownership review.
- Use Wiz and repository manager data to identify repeated vulnerable packages, risky child dependencies, direct public downloads, missing lockfiles, and unmanaged package ecosystems.
- Refine dependency cooldown windows by ecosystem and business need.
- Reduce exception volume by improving package availability, approved base images, and developer environment tooling.
- Publish recurring metrics and progress to engineering leadership.

Exit Criteria

- Supply chain controls are standard across active engineering teams.
- Citizen-developed tools have a practical path into approved source control and scanning.
- Exceptions are time-bound, reviewed, and trending down.
- Package risk, local development risk, and incident readiness are included in AppSec governance reporting.

Proactive Defense Workstreams

Sandboxed Development

Bayview should move high-risk development toward isolated environments such as GitHub Codespaces, stripped-down VMs, sandboxed build containers, or dedicated CI/CD runners. This reduces the damage when a malicious package executes during install, build, test, import, or runtime.

The roadmap should not rely on an immediate laptop-development ban before the alternative is ready. The practical sequence is:

1. Build a usable happy path.
2. Pilot it with high-risk teams.
3. Move package installation and deployment workflows into the sandbox.
4. Reduce secrets available on local laptops.
5. Restrict or prohibit local laptop development for Tier 1 projects.
6. Expand restrictions based on risk and readiness.

Developer Happy Path

Sandboxed development will only work if developers have the tools they need. Bayview should maintain standard environment definitions that include approved language runtimes, package manager configuration, IDE support, debugging tools, test tools, certificate trust, deployment utilities, and documented support.

Tool installation should have a clear request and approval process. Otherwise, developers will experience the sandbox as friction and try to return to unmanaged local workflows.

Local Laptop Development Restrictions

A complete ban on local laptop development is a major operational control and should be handled as a phased program rather than a surprise mandate. The roadmap direction should be clear: Tier 1 projects should stop relying on laptop-local development once Codespaces, stripped-down VMs, or equivalent sandboxed environments are ready.

After Tier 1 adoption is stable, Bayview should evaluate whether the local development ban should expand to additional project tiers based on credential exposure, data sensitivity, production access, and developer experience maturity.

Cleanroom, Hardened, and Backported Packages

Services such as Seal Security, Chainguard, or equivalent approved providers should be evaluated for:

- Building or sourcing packages from controlled cleanroom environments instead of relying directly on public registry artifacts from npm, PyPI, Maven, or other ecosystems.
- Reducing exposure when a child dependency is compromised.
- Providing hardened container images or packages with fewer unnecessary components.
- Providing backported CVE fixes when direct upstream upgrades are risky or time-consuming.

These services can reduce dependency compromise and vulnerability risk, but they do not fully protect against upstream source code tampering, malicious maintainer behavior, compromised CI/CD workflows, or attacks against Bayview's own source control. They should complement source control protections, CI/CD hardening, maintainer account security, code review, and incident response.

Repository Management

Bayview should use repository management platforms such as Nexus, JFrog Artifactory, or approved equivalents to improve package visibility and control.

Repository management should support:

- Approved package sources for each ecosystem.
- Version pinning or approved version ranges.
- Audit logs of package downloads.
- Visibility into packages used by developers and CI/CD.
- Blocking or alerting on known malicious, suspicious, unlicensed, or critically vulnerable packages.
- Dependency cooldowns for newly published versions.
- Stable-version defaults for production-bound software.

Package Risk Firewall

Bayview should evaluate Socket.dev or an equivalent install-time package-risk firewall as a complement to repository management. Repository managers control package routing, caching, access, and auditability. A package-risk firewall adds a policy decision before a package is downloaded or admitted into the internal package path.

This is most useful for npm, PyPI, Maven, NuGet, Go, RubyGems, Cargo, and other ecosystems where malicious packages, typosquats, dependency confusion, risky install scripts, abandoned packages, or unusual maintainer behavior can create risk before a traditional CVE exists.

Evaluation questions should include:

- Can the tool sit upstream of Artifactory or Nexus so public packages are checked before entering the internal repository cache?
- Can it protect developer workstations, Codespaces, sandboxed VMs, and CI/CD runners without creating fragile package manager configuration?
- Which ecosystems and package managers are fully supported for Bayview's current stack?
- Can policy distinguish between known malware, suspicious behavior, newly published packages, license issues, and ordinary CVEs?
- How are false positives, allowlists, emergency exceptions, audit logs, and developer experience handled?
- What telemetry leaves Bayview environments, and can usage data be governed consistently with internal privacy and vendor-risk requirements?

Dependency Cooldowns

Dependency cooldowns delay use of newly published package versions so the public ecosystem, security vendors, and maintainers have time to detect malicious releases.

Cooldowns are most useful when:

- Teams otherwise auto-update to the newest dependency version.
- A package has install scripts or build-time execution.
- A package has broad transitive reach.
- A package is used in CI/CD, developer tooling, or production infrastructure.

Bayview should start with npm, then expand to PyPI and other ecosystems where tooling supports it.

Containerization and Minimal Images

Containerization can reduce supply chain risk when it standardizes runtime dependencies and removes unnecessary packages. Teams should prefer minimal approved base images and avoid shipping build tools, shells, package managers, network tools, and debugging utilities in production images unless required.

Fewer binaries and fewer dependencies reduce vulnerability exposure and may limit what an attacker can do if a malicious package runs inside the container.

Reactive Defense Workstreams

Exposure Analysis

When a new campaign is reported, the SOC and AppSec teams should use Wiz Code, Wiz Mika or equivalent investigation capabilities, repository manager data, package-risk firewall logs, lockfiles, SBOMs, source control search, and CI/CD logs to determine whether Bayview uses the affected package, version, image, or workflow.

Indicator Hunting

Reactive detection should include searches for indicators of compromise such as:

- Suspicious hostnames or IP addresses.
- Unexpected files written during package install, build, import, or test.
- Unusual outbound connections from developer environments, CI/CD runners, containers, or build hosts.
- Unexpected package install scripts or import-time execution.
- Modified repository settings, workflow files, publishing tokens, or package registry credentials.
- New public repositories, unexpected forks, or unusual source control activity.

Credential Response

If a compromised package executed in an environment that had secrets, Bayview should assume those secrets may be exposed until proven otherwise.

Credential response should include:

- Identify what secrets were present in the affected environment.
- Rotate exposed or potentially exposed tokens.
- Revoke unused or over-scoped credentials.
- Review audit logs for use of the exposed credentials.
- Replace long-lived personal credentials with short-lived, scoped, environment-specific credentials where possible.

Rebuild and Recovery

Affected environments should be rebuilt from trusted sources when compromise is plausible. This includes developer sandboxes, CI/CD runners, build containers, and application images. Production redeployments should use clean package sources, approved versions, updated lockfiles, and validated build pipelines.

Metrics

- Percentage of Tier 1 and Tier 2 repositories with source-controlled manifests and lockfiles.
- Percentage of repositories using approved package sources.
- Number of direct public package downloads detected in CI/CD.
- Number of new package requests submitted, approved, denied, or excepted.
- Percentage of high-risk projects using sandboxed development environments.
- Number of long-lived developer secrets removed or replaced with scoped credentials.
- Percentage of Tier 1 applications using approved minimal or hardened base images.
- Number of critical package vulnerabilities remediated through upgrade, removal, replacement, hardened package, backported fix, or exception.
- Number of packages blocked or delayed by dependency cooldowns.
- Number of package-risk firewall blocks, warnings, allowlist overrides, and false-positive exceptions.
- Mean time to answer whether Bayview is affected by a reported package campaign.
- Mean time to rotate exposed credentials during supply chain incidents.
- Number of open exceptions by owner, age, and expiration date.

Open Decisions

- Which platform should be the primary approved package source for each ecosystem?

- Should Bayview pilot Socket.dev or an equivalent package-risk firewall, and should it run upstream of Artifactory/Nexus, downstream for developers and CI/CD, or both?
- Which ecosystems should receive dependency cooldowns first, and what cooldown window should apply?
- Which teams should pilot Codespaces, stripped-down VMs, or sandboxed build containers?
- What is the target date for restricting local laptop development for Tier 1 projects?
- Which secrets are allowed in developer environments, sandboxes, CI/CD runners, and package publish workflows?
- Which hardened package or container provider should Bayview evaluate first?
- Where will package approvals, exceptions, and risk acceptances be recorded?
- What response-time objective should SOC and AppSec use for exposure analysis during a public campaign?

References

- [Supply Chain Security](#)
- [Citizen Developers and Vibe Coders](#)
- [Dependency Cooldowns](#)
- [Akamai: Mini Shai-Hulud Worm Returns and Goes Public](#)
- [Microsoft: Shai-Hulud 2.0 Guidance](#)
- [ReversingLabs: Shai-Hulud Code Drop](#)
- [Seal Security](#)
- [Chainguard Images](#)
- [Socket.dev Firewall](#)
- [JFrog Artifactory](#)
- [Sonatype Nexus Repository](#)